

WEEKLY ASSESSMENT - 3

Python Programming Solutions Manual

1. Write a while loop that terminates when a user types "exit".

A `while True:` loop runs indefinitely until an explicit `break` statement is triggered based on a condition check verifying user input.

```
# Simulating user input loop
while True:
    user_input = input("Enter something (type 'exit' to quit): ")
    if user_input.strip().lower() == "exit":
        print("Loop terminated.")
        break
    print(f"You entered: {user_input}")
```

2. Create a dictionary of 5 students and their marks; print only those who passed (>35).

We iterate through a dictionary's key-value pairs using the `.items()` method and apply a basic conditional filter to isolate records matching our condition.

```
student_marks = {
    "Rahul": 42,
    "Sneha": 28,
    "Amit": 75,
    "Pooja": 33,
    "Vikram": 90
}

print("Students who passed (>35):")
for student, marks in student_marks.items():
    if marks > 35:
        print(f"{student}: {marks}")
```

Output:

```
Rahul: 42
Amit: 75
Vikram: 90
```

3. Use a list comprehension to square all numbers in a list.

List comprehensions provide a concise syntax for constructing lists out of existing iterable series structures.

```
numbers = [1, 2, 3, 4, 5]
squares = [num ** 2 for num in numbers]
print(squares)
```

Output:

```
[1, 4, 9, 16, 25]
```

4. How do you merge two dictionaries in Python 3.9+?

Python 3.9 introduced the merge operator `|` to combine pairs of dictionaries cleanly. Values from the right-hand side operand override overlapping keys from the left side.

```
dict1 = {'a': 1, 'b': 2}
dict2 = {'b': 99, 'c': 4}

# Using the merge operator
merged_dict = dict1 | dict2
print(merged_dict)
```

Output:

```
{'a': 1, 'b': 99, 'c': 4}
```

5. Write a script to find the frequency of each character in a string.

You can keep track of custom element occurrences using a regular empty dictionary or by importing Python's built-in `Counter` metric tool from the `collections` module.

```
text = "hello"
frequency = {}

for char in text:
    frequency[char] = frequency.get(char, 0) + 1

print(frequency)
```

Output:

```
{'h': 1, 'e': 1, 'l': 2, 'o': 1}
```

6. Demonstrate the enumerate() function in a for loop.

The `enumerate()` function adds a counter index tracking sequence placement to an iterable, returning pairs containing the item position and the element itself.

```
languages = ["Python", "Java", "C++"]

for index, lang in enumerate(languages):
    print(f"Index {index}: {lang}")
```

Output:

```
Index 0: Python
Index 1: Java
Index 2: C++
```

7. Write a program to find the intersection of two sets.

Set intersection isolates common entries existing across multiple sets using either the `.intersection()` method or the bitwise `&` operator.

```
set_a = {1, 2, 3, 4}
set_b = {3, 4, 5, 6}

# Method 1: Using & operator
result = set_a & set_b
print(result)
```

Output:

```
{3, 4}
```

8. Use zip() to combine two lists into a dictionary.

The `zip()` function pairs corresponding index entries from collections together, allowing the output to be mapped into structured key-value maps via the `dict()` constructor.

```
keys = ["name", "age", "role"]
values = ["Ananya", 21, "Developer"]

combined_dict = dict(zip(keys, values))
print(combined_dict)
```

Output:

```
{'name': 'Ananya', 'age': 21, 'role': 'Developer'}
```

9. Create a 3x3 matrix using nested lists and access the middle element.

A multi-dimensional array pattern can be established via embedded tracking components. The middle item sits at standard zero-based tracking position `[1][1]`.

```
matrix = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]  
  
middle_element = matrix[1][1]  
print("Middle Element:", middle_element)
```

Output:

```
Middle Element: 5
```

10. Write a script to sort a dictionary by its values.

You can use the built-in `sorted()` tool, applying an anonymous lambda sorting key expression targeting dictionary item evaluation markers to organize values cleanly.

```
scores = {"Math": 85, "Science": 92, "English": 78}  
  
# Sorting items by value element  
sorted_items = sorted(scores.items(), key=lambda item: item[1])  
sorted_dict = dict(sorted_items)  
  
print(sorted_dict)
```

Output:

```
{'English': 78, 'Math': 85, 'Science': 92}
```